3

CONWAY'S GAME OF LIFE

You can use a computer to study a system by creating a mathematical model for that system, writing a program to represent the model, and then letting the model evolve over time. There are many kinds of computer simulations, but I'll focus on a famous one called Conway's Game of Life, the work of the British mathematician John Conway. The Game of Life is an example of a *cellular automaton*, a collection of colored cells on a grid that evolve through a number of time steps according to a set of rules defining the states of neighboring cells.

In this project, you'll create an $N \times N$ grid of cells and simulate the evolution of the system over time by applying the rules of Conway's Game of Life. You'll display the state of the game at each time step and provide ways to save the output to a file. You'll set the initial condition of the system to either a random distribution or a predesigned pattern.

This simulation consists of the following components:

- A property defined in one- or two-dimensional space
- A mathematical rule to change this property for each step in the simulation

- A way to display or capture the state of the system as it evolves

The cells in Conway's Game of Life can be either ON or OFF. The game starts with an initial condition, in which each cell is assigned one of these states. Then, mathematical rules determine how each cell's state will change over time. The amazing thing about Conway's Game of Life is that with just four simple rules the system evolves to produce patterns that behave in incredibly complex ways, almost as if they were alive. Patterns include "gliders" that slide across the grid, "blinkers" that flash on and off, and even replicating patterns.

Of course, the philosophical implications of this game are also significant because they suggest that complex structures can evolve from simple rules without following any sort of preset pattern.

Here are some of the main concepts covered in this project:

- Using `matplotlib imshow` to represent a 2D grid of data
- Using `matplotlib` for animation
- Using the `numpy` array
- Using the `%` operator for boundary conditions
- Setting up a random distribution of values

How It Works

Because the Game of Life is built on a grid of nine squares, every cell has eight neighboring cells, as shown in **Figure 3-1**. A given cell (*i*, *j*) in the simulation is accessed on a grid [*i*][*j*], where *i* and *j* are the row and column indices, respectively. The value of a given cell at a given instant of time depends on the state of its neighbors at the previous time step.

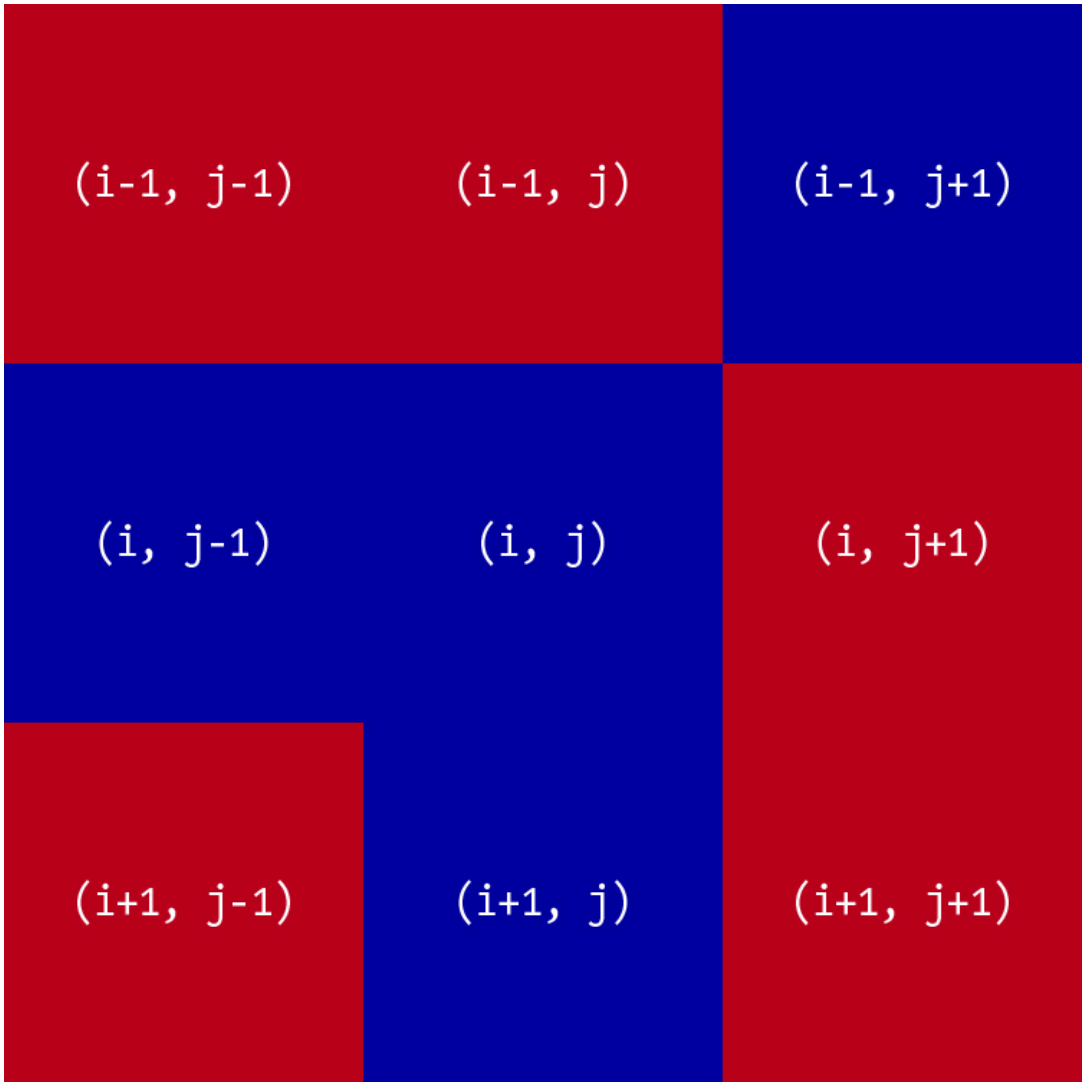


Figure 3-1: *A central cell* (i, j) *with eight neighboring cells*

Conway's Game of Life has four rules:

1. If a cell is ON and has fewer than two neighbors that are ON, it turns OFF.
2. If a cell is ON and has either two or three neighbors that are ON, it remains ON.
3. If a cell is ON and has more than three neighbors that are ON, it turns OFF.
4. If a cell is OFF and has exactly three neighbors that are ON, it turns ON.

These rules are meant to mirror some basic ways that a group of organisms might fare over time: underpopulation and overpopulation kill cells by turning a cell OFF when it has fewer than two neighbors or more than three, but when the population is balanced, cells stay ON and reproduce by turning another cell from OFF to ON.

I said that each cell has eight neighboring cells, but what about cells at the edge of the grid? Which cells are their neighbors? To answer this question, you need to think about *boundary conditions*, the rules that govern what happens to cells at the edges or boundaries of the grid. I'll address this question by using *toroidal boundary conditions*, meaning that the square grid wraps around as if its shape were a torus. As shown in **Figure 3-2**, the grid is first warped so that its horizontal edges (A and B) join to form a cylinder, and then the cylinder's vertical edges (C and D) are joined to form a torus. Once the torus has been formed, all cells have neighbors because the whole space has no edge.

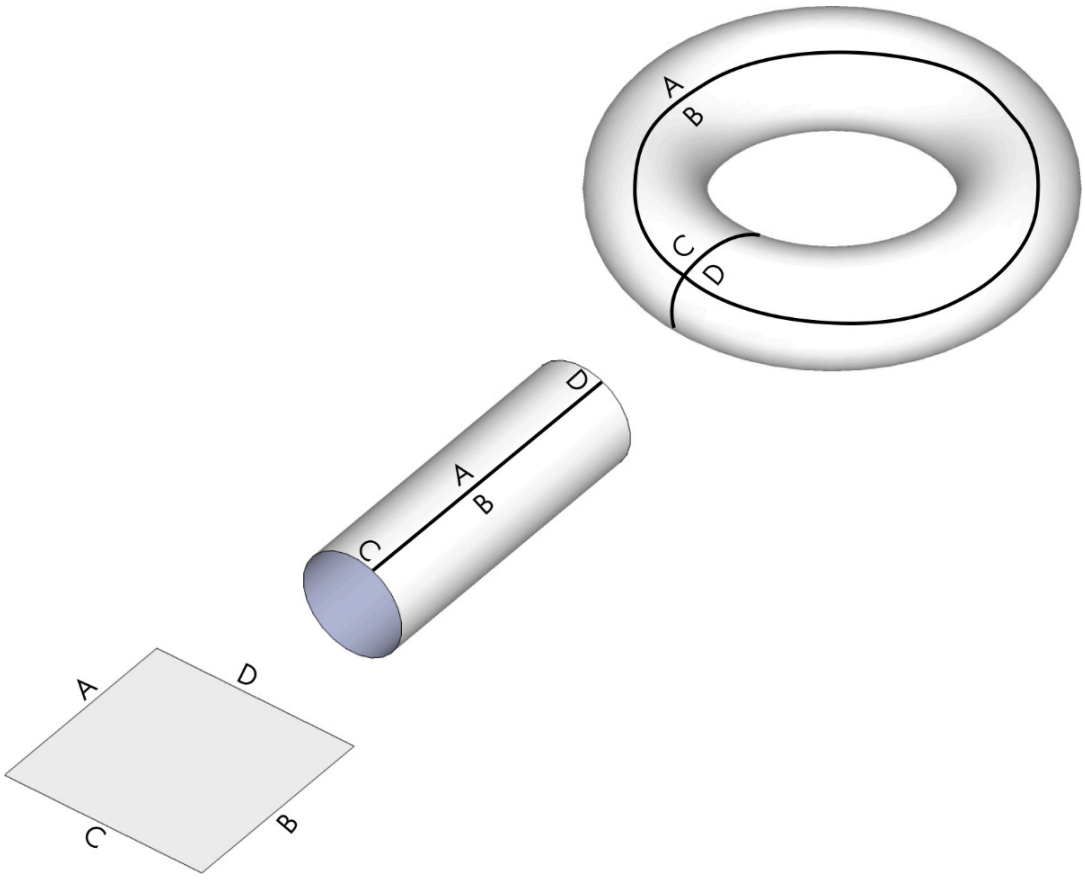


Figure 3-2: *A conceptual visualization of toroidal boundary conditions*

Toroidal boundary conditions are common in 2D simulations and games. For example, the game *Pac-Man* uses them. If you go off the top of the screen, you reappear on the bottom. If you go off the left side of the screen, you reappear on the right side. You'll follow the same logic for the Game of Life simulation: for the cell in the top-left corner of the grid, for example, the neighbor directly above it will be the cell in the bottom-left corner, and the neighbor directly to the left of it will be in the cell in the top-right corner.

Here's a description of the algorithm you'll use to apply the four rules and run the simulation:

1. Initialize the cells in the grid.
2. 2. At each time step in the simulation, for each cell (*i*, *j*) in the grid, do the following:
    a. Update the value of cell (*i*, *j*) based on its neighbors, taking into account the boundary conditions.
    b. Update the display of grid values.

Requirements

You'll use `numpy` arrays and the `matplotlib` library to display the simulation output, and you'll use the `matplotlib` `animation` module to update the simulation.

The Code

We'll examine key aspects of the program piece by piece, including how to represent the simulation grid using `numpy` and `matplotlib`, how to set the initial conditions for the simulation, how to account for toroidal boundary conditions, and how to implement the Game of Life rules. We'll also look at the program's `main()` function, which sends command line arguments to the program and sets the simulation into motion. To see the full project code, skip ahead to **"The Complete**

Code" on page 56. You can also download the code from *https://github.com/mkvenkit/pp2e/tree/main/conway*.

Representing the Grid

To represent whether a cell is alive (ON) or dead (OFF) on the grid, you'll use the values `255` and `0` for ON and OFF, respectively. You'll display the current state of the grid using the `imshow()` method in `matplotlib`, which represents a matrix of numbers as an image. To get a feel for how it works, let's try a simple example inside the Python interpreter. Enter the following:

```
>>> import numpy as np

>>> import matplotlib.pyplot as plt

❶ >>> x = np.array([[0, 0, 255], [255, 255, 0], [0, 255, 0]])

❷ >>> plt.imshow(x, interpolation='nearest')

>>> plt.show()
```

You define a 2D `numpy` array of shape (3, 3) ❶, meaning the array has three rows and three columns. Each element of the array is an integer value. You then use the `plt.imshow()` method to display this matrix of values as an image ❷, passing in the interpolation option as `'nearest'` to get sharp edges for the cells (otherwise, they'd be fuzzy). Figure 3-3 shows the output of this code.

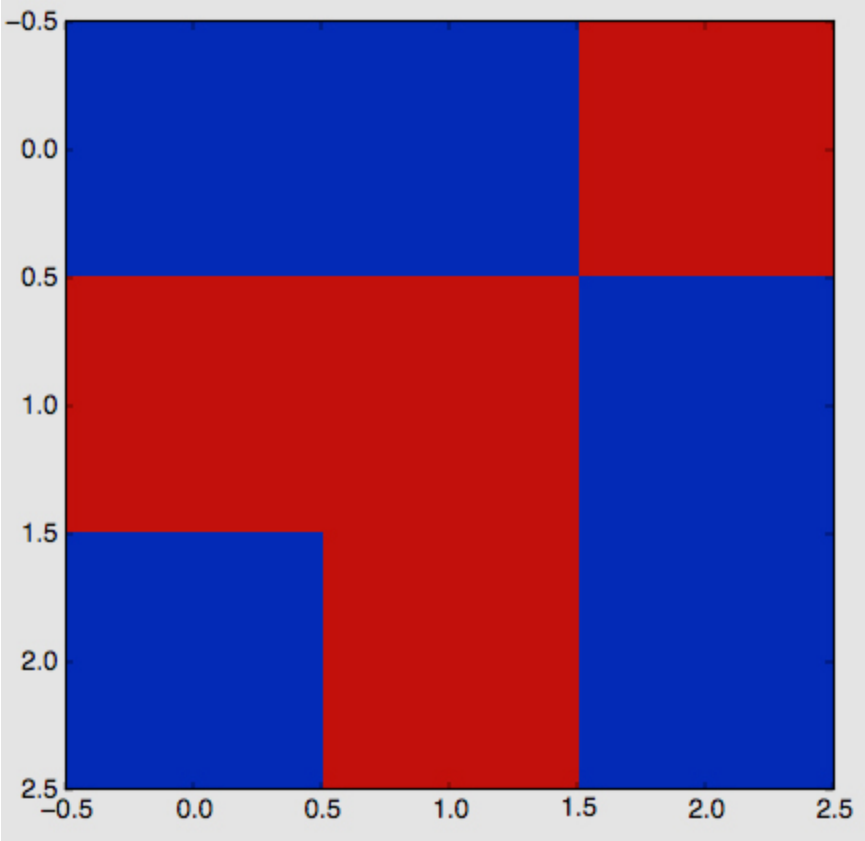


Figure 3-3: *Displaying a grid of values*

Notice that squares with a value of `0` (OFF) are given a darker color, while squares with a value of `255` (ON) are given a lighter color.

Setting the Initial Conditions

To begin the simulation, set an initial state for each cell in the 2D grid. You can use a random distribution of ON and OFF cells and see what kinds of patterns emerge, or you can add some specific patterns and see how they evolve. We'll look at both approaches.

For a random initial state, use the `choice()` method from the `random` module in `numpy`. Enter the following in the Python interpreter to see how it works:

```
>>> np.random.choice([0, 255], 4*4, p=[0.1, 0.9]).reshape(4, 4)
```

The output will be something like this:

```
array([[255, 255, 255, 255],
       [255, 255, 255, 255],
       [255, 255, 255, 255],
       [255, 255, 255, 0]])
```

`np.random.choice()` chooses a value from the given list `[0, 255]`, with the probability of the appearance of each value given in the parameter `p=[0.1, 0.9]`. Here you ask for 0 to appear with a probability of 0.1 (or 10 percent) and for 255 to appear with a probability of 90 percent. (The two values in `p` must add up to 1.) The `choice()` method creates a one-dimensional array, in this case with 16 values (specified with `4*4`). You use `reshape()` to make it a two-dimensional array with four rows and four columns.

To set up the initial condition to match a particular pattern instead of just filling in a random set of values, first use `np.zeros()` to initialize the grid with all zeros:

```
grid = np.zeros(N*N).reshape(N, N)
```

This creates an $N \times N$ array of zeros. Now define a function to add a pattern at a particular row and column in the grid:

```
def addGlider(i, j, grid):
    """adds a glider with top left cell at (i, j)"""
❶   glider = np.array([[0, 0, 255],
                       [255, 0, 255],
```

```
                [0, 255, 255]])

❷ grid[i:i+3, j:j+3] = glider
```

You define the glider pattern (an observed pattern that moves steadily across the grid) using a `numpy` array of shape (3, 3) ❶. Then you use the `numpy` slice operation to copy the `glider` array into the simulation's two-dimensional grid ❷, with the pattern's top-left corner placed at the coordinates you specify as `i` and `j`.

Now you can add a glider pattern to the grid of zeros with a call to the `addGlider()` function you just defined:

```
addGlider(1, 1, grid)
```

You specify coordinates of (1, 1) to add the glider near the top-left corner of the grid, which is (0, 0). Note that for `grid[i, j]`, `i` starts at `0` and runs from top to bottom, and `j` starts at `0` and runs from left to right.

Enforcing the Boundary Conditions

Now we can think about how to implement the toroidal boundary conditions. First, let's see what happens at the right edge of a grid of size $N \times N$. The cell at the end of row i is accessed as `grid[i, N-1]`. Its neighbor to the right is `grid[i, N]`, but according to the toroidal boundary conditions, the value accessed as `grid[i, N]` should be replaced by `grid[i, 0]`. Here's one way to do that:

```
if j == N-1:

    right = grid[i, 0]

else:
```

```
right = grid[i, j+1]
```

Of course, you'd need to apply similar boundary conditions to the left, top, and bottom sides of the grid, but doing so would require adding a lot more code because each of the four edges of the grid would need to be tested. A much more compact way to implement the boundary conditions is with Python's modulus (`%`) operator, demonstrated here in the Python interpreter:

```
>>> N = 16
>>> i1 = 14
>>> i2 = 15
>>> (i1+1)%N
15
>>> (i2+1)%N
0
```

As you can see, the `%` operator gives the remainder for the integer division by `N`. In this example, `15%16` yields `15`, but `16%16` yields `0`. You can use the `%` operator to make the values wrap around at the right edge by rewriting the grid access code like this:

```
right = grid[i, (j+1)%N]
```

Now when a cell is on the edge of the grid (in other words, when `j = N-1`), asking for the cell to the right with this method will give you `(j+1)%N`, which sets `j` back to `0`, making the right side of the grid wrap to the left side. When you do the same for the bottom of the grid, it wraps around to the top:

```
bottom = grid[(i+1)%N, j]
```

Implementing the Rules

The rules of the Game of Life are based on the number of neighboring cells that are ON or OFF. To simplify the application of these rules, you can just calculate the total number of neighboring cells in the ON state. Because ON corresponds to a value of 255, simply sum the values of all the neighbors and divide by 255 to get the number of ON cells. Here's the relevant code:

```
total = int((grid[i, (j-1)%N] + grid[i, (j+1)%N] +
        grid[(i-1)%N, j] + grid[(i+1)%N, j] +
        grid[(i-1)%N, (j-1)%N] + grid[(i-1)%N, (j+1)%N] +
        grid[(i+1)%N, (j-1)%N] + grid[(i+1)%N, (j+1)%N])/255)
```

For any given cell (*i*, *j*), you sum the value of its eight neighbors, using the `%` operator to account for toroidal boundary conditions. Dividing the result by 255 gives you the number of neighbors that are ON, which you store in the variable `total`. Now you can use `total` to apply the Game of Life rules:

```
# apply Conway's rules
if grid[i, j] == ON:
  ❶ if (total < 2) or (total > 3):
        newGrid[i, j] = OFF
else:
```

```
❷ if total == 3:
    newGrid[i, j] = ON
```

Any cell that is ON is turned OFF if it has fewer than two neighbors that are ON or if it has more than three neighbors that are ON ❶. The code in the `else` branch applies only to OFF cells: a cell is turned ON if exactly three neighbors are ON ❷. The changes are applied to the corresponding cells in `newGrid`, which starts as a copy of `grid`. Once every cell has been evaluated and updated, `newGrid` contains the data for displaying the next time step in the simulation. You can't make changes directly to `grid`, or the states of the cells would keep changing as you try to evaluate them.

Sending Command Line Arguments to the Program

Now you can begin writing the `main()` function of the simulation, which starts by sending command line arguments to the program:

```
def main():
    # command line arguments are in sys.argv[1], sys.argv[2], ...
    # sys.argv[0] is the script name and can be ignored
    # parse arguments
❶ parser = argparse.ArgumentParser(description="Runs Conway's
Game of Life
                                   simulation.")
    # add arguments
❷ parser.add_argument('--grid-size', dest='N', required=False)
```

```
❸ parser.add_argument('--interval', dest='interval', required=False)

❹ parser.add_argument('--glider', action='store_true', required=False)

  args = parser.parse_args()
```

You create an `argparse.ArgumentParser` object to add command line options to the code ❶, and then you add various options to it in the following lines. The option at ❷ specifies the simulation grid size *N*, and the option at ❸ sets the animation update interval in milliseconds. You also create an option to start the simulation with a glider pattern ❹. If this option isn't set, the simulation will start with random ON and OFF values.

Initializing the Simulation

Continuing through the `main()` function, you come to the following section, which initializes the simulation:

```
  # set grid size

❶ N = 100

  # set animation update interval

❷ updateInterval = 50

  if args.interval:

     updateInterval = int(args.interval)

  # declare grid

  grid = np.array([])

  # check if "glider" demo flag is specified
```

```
❸ if args.glider:
    grid = np.zeros(N*N).reshape(N, N)
    addGlider(1, 1, grid)
❹ else:
    # set N if specified and valid
    if args.N and int(args.N) > 8:
       N = int(args.N)
    # populate grid with random on/off - more off than on
    grid = randomGrid(N)
```

This portion of the code applies any parameters called at the command line, once the command line options have been parsed. First, a default grid size of 100×100 cells ❶ and an update interval of 50 milliseconds ❷ are set, in case these options aren't set at the command line. Then you set up the initial conditions, either a random pattern by default ❹ or a glider pattern ❸.

Finally, you set up the animation:

```
  # set up the animation
❶ fig, ax = plt.subplots()
  img = ax.imshow(grid, interpolation='nearest')
❷ ani = animation.FuncAnimation(fig, update, fargs=(img, grid, N, ),
                                interval=updateInterval,
```

```
                save_count=50)

    plt.show()
```

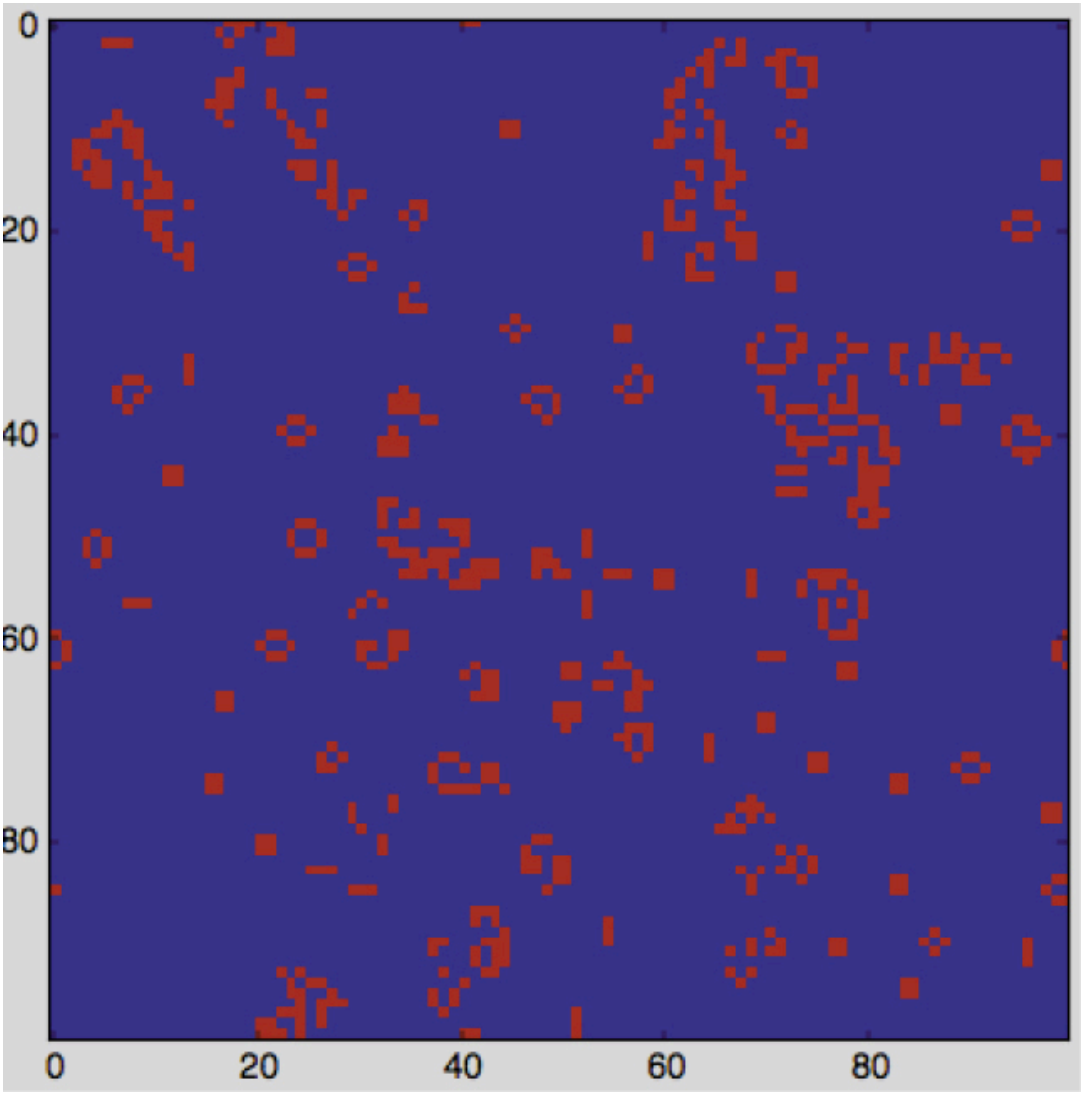
Still within the `main()` function, you configure the `matplotlib` plot and animation parameters ❶. Then you set `animation.FuncAnimation()` to regularly call the function `update()`, defined earlier in the program, which updates the grid according to the rules of Conway's Game of Life using toroidal boundary conditions ❷.

Running the Game of Life Simulation

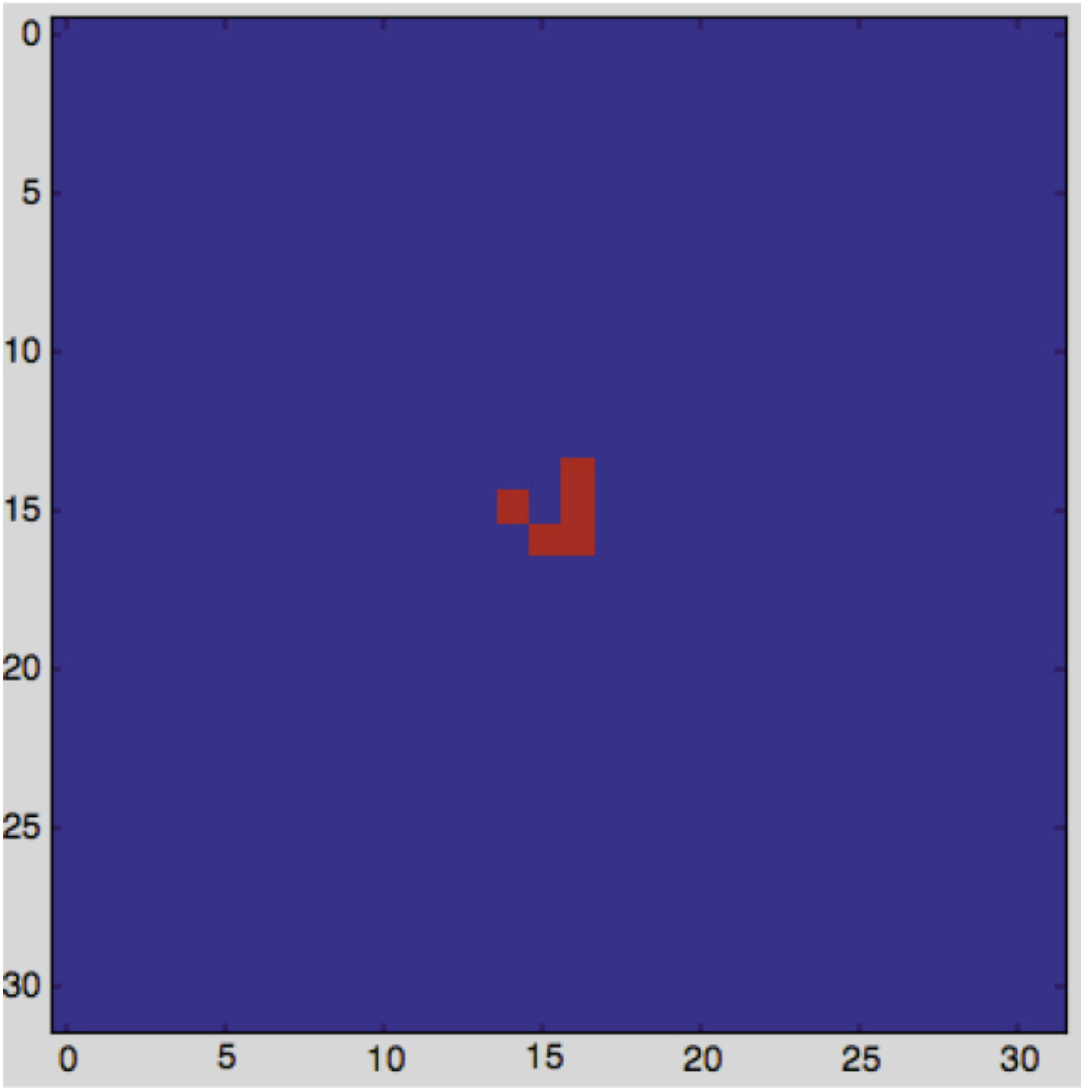
Now run the code:

```
$ python conway.py
```

This uses the default parameters for the simulation: a grid of 100×100 cells and an update interval of 50 milliseconds. As you watch the simulation, you'll see how it progresses to create and sustain various patterns over time, as in **Figure 3-4 (a) and (b)**.



(a)



(b)

Figure 3-4: *The Game of Life in progress*

Figure 3-5 shows some of the patterns to look for in the simulation. Besides the glider, look for a three-cell blinker and static patterns such as a block or loaf shape.

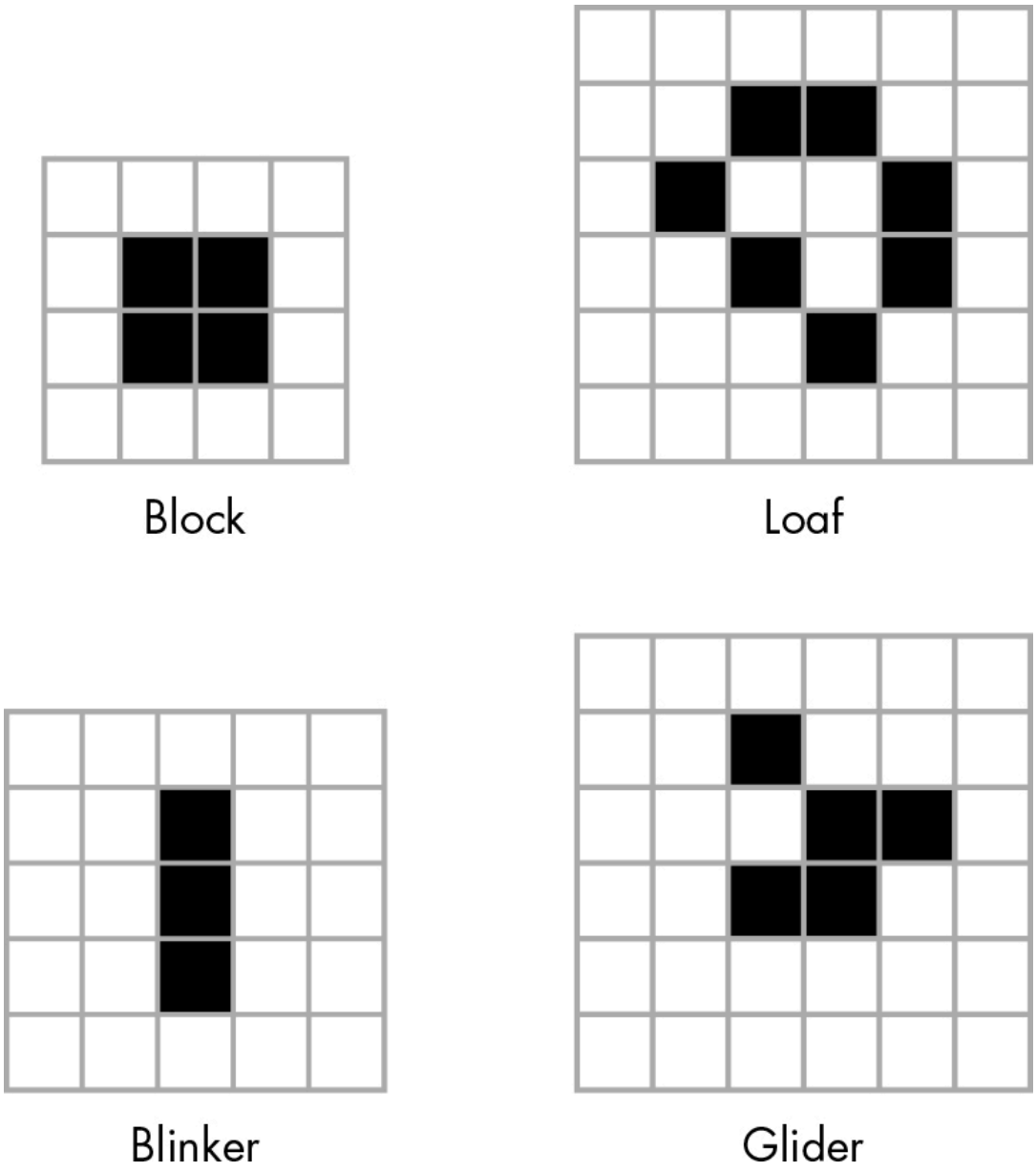


Figure 3-5: *Some patterns in the Game of Life*

Now change things up a bit by running the simulation with these parameters:

```
$ python conway.py --grid-size 32 --interval 500 --glider
```

This creates a simulation grid of 32×32, updates the animation every 500 milliseconds, and uses the initial glider pattern shown in the bottom right of **Figure 3-5**.

Summary

In this project, you explored Conway's Game of Life. You learned how to set up a basic computer simulation based on some rules and how to use `matplotlib` to visualize the state of the system as it evolves.

My implementation of Conway's Game of Life emphasizes simplicity over performance. You can speed up the computations in the Game of Life in many different ways, and a tremendous amount of research has been done on how to do this. You'll find a lot of this research through a quick internet search.

Experiments!

Here are some ways to experiment further with Conway's Game of Life:

1. 1. Write an `addGosperGun()` method to add the pattern shown in Figure 3-6 to the grid. This pattern is called the *Gosper Glider Gun*. Run the simulation and observe what the gun does.

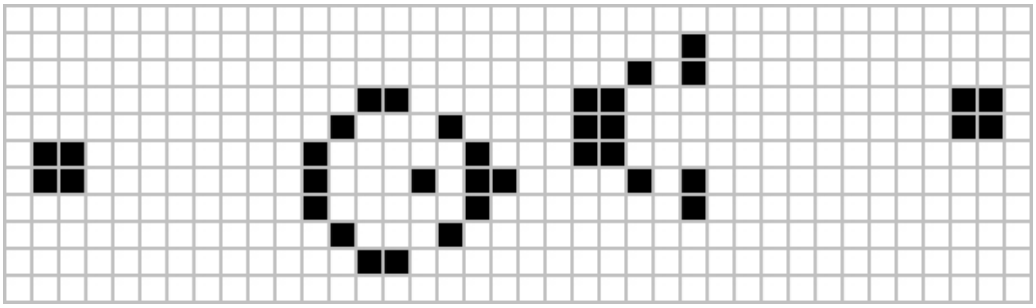


Figure 3-6: *The Gosper Glider Gun*

2. 2. Write a `readPattern()` method that reads in an initial pattern from a text file and uses it to set the initial conditions for the simulation. You can use Python methods such as `open` and `file.read` to do this. Here's a suggested format for the input file:

```
8
0 0 0 255...
```

The first line of the file defines *N*, and the rest of the file is just *N*×*N* integers (0 or 255) separated by whitespace. This exploration will help you study how any given pattern evolves with the rules of the Game of Life. Add a `--pattern-file` command line option to use this file while running the program.

The Complete Code

Here's the complete code for the Game of Life project:

```
"""

conway.py

A simple Python/matplotlib implementation of Conway's Game of Life.

Author: Mahesh Venkitachalam

"""

import sys, argparse

import numpy as np

import matplotlib.pyplot as plt

import matplotlib.animation as animation

def randomGrid(N):

    """returns a grid of NxN random values"""

    return np.random.choice([255, 0], N*N, p=[0.2, 0.8]).reshape(N, N)

def addGlider(i, j, grid):
```

```
    """adds a glider with top left cell at (i, j)"""
    glider = np.array([[0,    0, 255],
                       [255,  0, 255],
                       [0,  255, 255]])
    grid[i:i+3, j:j+3] = glider
def update(frameNum, img, grid, N):
    # copy grid since we require 8 neighbors for calculation
    # and we go line by line
    newGrid = grid.copy()
    for i in range(N):
        for j in range(N):
            # compute 8-neighbor sum
            # using toroidal boundary conditions - x and y wrap around
            # so that the simulation takes place on a toroidal surface
            total = int((grid[i, (j-1)%N] + grid[i, (j+1)%N] +
                         grid[(i-1)%N, j] + grid[(i+1)%N, j] +
                         grid[(i-1)%N, (j-1)%N] + grid[(i-1)%N, (j+1)%N] +
                         grid[(i+1)%N, (j-1)%N] + grid[(i+1)%N, (j+1)%N])/255)
            # apply Conway's rules
```

```
            if grid[i, j]  == 255:
                if (total < 2) or (total > 3):
                    newGrid[i, j] = 0
            else:
                if total == 3:
                    newGrid[i, j] = 255
    # update data
    img.set_data(newGrid)
    grid[:] = newGrid[:]
    # need to return a tuple here, since this callback
    # function needs to return an iterable
    return img,
# main() function
def main():
    # command line args are in sys.argv[1], sys.argv[2]...
    # sys.argv[0] is the script name itself and can be ignored
    # parse arguments
    parser = argparse.ArgumentParser(description="Runs Conway's
Game of Life
```

```
                        simulation.")

    # add arguments

    parser.add_argument('--grid-size', dest='N', required=False)

    parser.add_argument('--interval', dest='interval', required=False)

    parser.add_argument('--glider', action='store_true', required=False)

    parser.add_argument('--gosper', action='store_true', required=False)

    args = parser.parse_args()

    # set grid size

    N = 100

    # set animation update interval

    updateInterval = 50

    if args.interval:

        updateInterval = int(args.interval)

    # declare grid

    grid = np.array([])

    # check if "glider" demo flag is specified

    if args.glider:

        grid = np.zeros(N*N).reshape(N, N)

        addGlider(1, 1, grid)
```

```
    elif args.gosper:
        grid = np.zeros(N*N).reshape(N, N)
        addGosperGliderGun(10, 10, grid)
    else:
        # set N if specified and valid
        if args.N and int(args.N) > 8:
            N = int(args.N)
        # populate grid with random on/off - more off than on
        grid = randomGrid(N)
    # set up animation
    fig, ax = plt.subplots()
    img = ax.imshow(grid, interpolation='nearest')
    ani = animation.FuncAnimation(fig, update, fargs=(img, grid, N, ),
                                  frames = 10,
                                  interval=updateInterval)
    plt.show()
# call main
if __name__ == '__main__':
```

```
main()
```
